

Numerikus Számítások

2. előadás

Csókás Eszter

Szegedi Tudományegyetem
Informatika Intézet
Számítógépes Optimalizálási Tanszék



Számábrázolási formátumok [ismétlés]

Számábrázolás

- fixpontos számábrázolás:

- úgy tudunk törtszámokat kifejezni, hogy a számba egy bináris (kettedes) pontot "képzünk"
- a pont mozgását korrigáljuk eltolással, amikor műveleteket végzünk el
- bizonyos mértékben segít, de az alapvető problémán nem:
a számok nagysága tág határok között mozognak

- lebegőpontos számábrázolás:

- az ábrázolás pontosságát a szám tényleges nagyságához kellene igazítani
- azaz egy pontosság (pl: "kéttizedes pontosságú szám") nem mond semmit a szám nagyságának ismerete nélkül
- ezt a problémát a matematika a normál alak használatával hidalja át
- az informatika, vagy sokkal inkább számítástechnika világában pedig a lebegőpontos számábrázolást használjuk

A tört számok világa

Gondoljuk át a következő problémákat:

- számítógépen egy törtszámot véges számú biten tudunk ábrázolni és ennek az ábrázolásnak van egy véges pontossága, számábrázolási nagysága; míg a törtszám, mint matematikai értelemben vett valós szám, ezt nem tartalmazza
- véges törtrészü tizedes törtek a kettes számrendszerben történő ábrázolás miatt nem biztos, hogy véges bitszámon lesz ábrázolható
→ *a 0.1 véges tört a tízes számrendszerben, de mi a helyzet a kettes számrendszerben?*

Emlékszünk erre az egyszerű példára múlt óráról? Van már sejtésünk, hogy miért hamis a kifejezés?

$$x1 = 0.3$$

$$x2 = 0.1 + 0.1 + 0.1$$

$$x1 == x2$$

Az egész típus

Az ábrázolható számok

bit	név	tartomány	tiz.j.
8	byte, octet	előjeles: $(-2^7) = -128$ -tól $(2^7 - 1) = +127$ -ig	3
		előjel nélkül: 0-tól $(2^8 - 1) = 255$ -ig	3
16	halfword, word	előjeles: $(-2^{15}) = -32768$ -tól $(2^{15} - 1) = +32767$ -ig	5
		előjel nélkül: 0-tól $(2^{16} - 1) = 65535$ -ig	5
32	word, doubleword, longwordn	előjeles: $(-2^{31}) = -2147483648$ -tól $(2^{31} - 1) = 2147483647$ -ig	10
		előjel nélkül: 0-tól $(2^{32} - 1) = 4294967295$ -ig	10
64	doubleword, longword, long long, quad, quadword	előjeles: (-2^{63}) -tól $(2^{63} - 1)$ -ig	19
		előjel nélkül: 0-tól $(2^{64} - 1)$ -ig	20
128		előjeles: (-2^{127}) -tól $(2^{127} - 1)$ -ig	39
		előjel nélkül: 0-tól $(2^{128} - 1)$ -ig	39

Előjeles számábrázolás

Mi van az előjelekkel? A ”-” szimbólum nem fér bele a ”0-1” karakterek közé

- előjeles abszolút érték:
 - balról az első bit az előjel: 0, ha a szám pozitív; 1, ha a szám negatív
 - a 0 szám kétféleképpen ábrázolható: 10000000, 00000000
 - a legkisebb szám a -127, míg a legnagyobb a 127
- egyes komplementens:
 - az első bit az előjel: 0, ha a szám pozitív; 1, ha a szám negatív
 - képzési szabály: egy szám negatívszorosát úgy kapjuk meg, hogy minden bináris számjegyet invertálunk ($0 \rightarrow 1, 1 \rightarrow 0$)
 - 0 kétféleképpen ábrázolható: 00000000, 11111111

Előjeles számábrázolás

Mi van az előjelekkel? A ”-” szimbólum nem fér bele a ”0-1” karakterek közé

- kettes komplementum:

- az első bit az előjel: 0, ha a szám pozitív; 1, ha a szám negatív
 - egy negatív szám úgy kapható meg, hogy az abszolút értékének egyes komplementumához hozzáadunk 1-et (csak az eredeti számú bitet őrizzük meg)
 - a nulla egyértelműen ábrázolható: 0 = 00000000
 - továbbá pl.: $127 = 01111111$, $-128 = 10000000$, $-1 = 11111111$
-
- Feladat: Nézzük meg a 43_{10} és a -123_{10} kettes komplementumát!

Feladat: Nézzük meg a 43_{10} és a -123_{10} kettes komplementjét!

$43_{10} \Rightarrow$ poz. szám
önmaga

$$k_3 = 00101011_2$$

$-123_{10} \Rightarrow$ negativ Zahlen

① $123_{10} = 01111011_2$

② Types complexes:

1 0000 1 00

③ Kettes komp.:

$$\begin{array}{r} 10000100 \\ + \quad \quad \quad 1 \\ \hline 10000101_2 \end{array}$$

Előjeles számábrázolás

Másik módszer kettes komplementes képzésre:

A szám értékét (A) vonjuk ki binárisan 2^n -ből:

$$2^n - A,$$

ahol n a bitek száma. Ebből a képletből látszódik, hogy a kettes komplementes számábrázolás bitszámhoz kötött.

Példa: -5 képzése 4 bitre: $n = 4$ $2^4 - 5 \rightarrow 10000 - 0101 = 1011$

Előjel-kiterjesztés

Előjel-kiterjesztéskor a bináris számjegyek csökkentése vagy növelése történik:

- az érték nem változik!
- *jelentősége*: kézenfekvő eset, amikor különböző méretű adatformátumok között hajtunk végre konverziót (8 bites bájt $\rightarrow$ 32 bites szó)
- pozitív számok esetén: a kiterjesztés egyszerű, hiszen tudjuk, hogy egy szám elé tetszőleges darab 0-t írva, a szám nem változik
- negatív számok esetén: a kiterjesztést gondoljuk végig!

$$101_2 = 000101_2$$

Előjel-kiterjesztés negatív számok esetén

Az A számunkat szeretnénk n bitről N bitesre átírni úgy, hogy $N > n$, ekkor a következő összefüggések igazak:

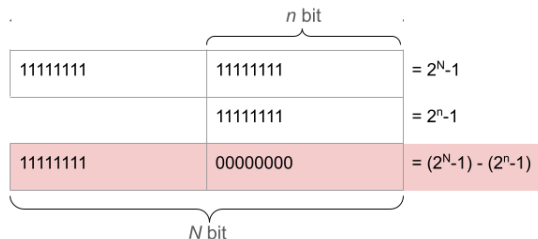
$$x = 2^n - |A|, \quad y = 2^N - |A|,$$

tehát a kettes komponens alapján látjuk, hogy y értékét szeretnénk meghatározni x -ből.

n bit		
11111111	11111111	$= 2^N - 1$
	11111111	$= 2^n - 1$
11111111	00000000	$= (2^N - 1) - (2^n - 1)$

$$y = 2^N - |A| = 2^N - \underbrace{2^n}_{0} + 2^n - |A| = (2^N - 1) - (2^n - 1) + \underbrace{2^n - |A|}_x = (2^N - 1) - (2^n - 1) + x$$

Előjel-kiterjesztés negatív számok esetén



$$y = 2^N - |A| = 2^N - 2^n + 2^n - |A| = (2^N - 1) - (2^n - 1) + 2^n - |A|$$

$$y = (2^N - 1) - (2^n - 1) + x$$

Mit látunk ebből?

- negatív szám esetén a szám elejét egyesekkel kell feltölteni

Általánosítás

Mivel egy pozitív szám előjel bitjének értéke mindig 0, negatív szám esetén pedig ez 1, ezért az előbbieket általánosíthatjuk:

előjel-kiterjesztés során a szám saját előjelbitjeinek értékével kell feltölteni a szám hiányzó bitjeit

Általánosítás

előjel-kiterjesztés során a szám saját előjelbitjeinek értékével kell feltölteni a szám hiányzó bitjeit

Nézzük a fordítottját, azaz a számábrázolási hossz csökkentése során a következőket fogalmazhatjuk meg:

A szám elejéről elhagyhatjuk (levághatjuk) a biteket addig, amíg

- 1. a levágott bitek azonosak (csupa nullák vagy csupa egyesek) ÉS**
- 2. megegyeznek a lerövidült szám elején lévő bittel**

Példa, feladat

	8 biten	12 biten	16 biten
+5	0 000 0101	0000 0 000 0101	0000 0000 0 000 0101
-5	<i>1 1 1 1 1 0 1 1</i>	<i>1 1 1 1 1 1 1 1 1 0 1 1</i>	<i>1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1</i>
+20	0 001 0100	0000 0 001 0100	0000 0000 0 001 0100
-20	1 110 1100		

A valós típus

Fixpontos számábrázolás

Az ábrázolt számba beleképzelünk egy kettedes pontot és innentől a számmal ugyanúgy végezzük el a műveleteket, mint az egész számokkal.

- a helyértékeken balról jobbra a súlyozó faktorok folyamatos feleződés teljesül az egészekre (16, 8, 4, 2, 1)
- és ez folytatódik a törtrészben is ($\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}$ stb.)

$$\begin{array}{c|c|c|c} 2^1 & 2^0 & 2^{-1} & 2^{-2} \\ \hline 2 & 5 & 5 & 7 \end{array}$$

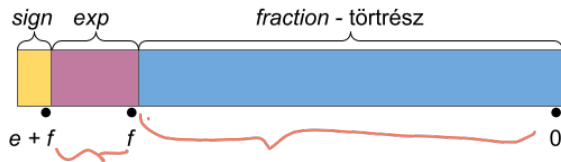
Úgy képzeljük el egy 16 biten ábrázolt (8 bites egész rész és 8 bites törtrészt tartalmazó) fixpontos törtszámot, mintha a szám helyett az ő 256-szorosát ábrázoltuk volna, de amikor értelmezzük a számot, akkor képzeletben elosztjuk 256-tal a kettedes pont odagondolásával.

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

A lebegőpontos számábrázolás

$$1,2 \cdot 10^2 ; 5,6 \cdot 10^{-6}$$

A matematikai normál alak fogalmához kapcsolódik. A számok ábrázolása a következőképpen néz ki:



A szám értelmezése pedig:

$$x = \pm m \cdot 2^k,$$

ahol az m mantissza, ami nem azonos a törtrésszel;

a k pedig a karakterisztika, ami értékét tekintve nem azonos az *exp* mező értékével.

A lebegőpontos számábrázolás

$$x = \pm m \cdot 2^k,$$

A mantissza (m):

- a mantissza az $[1.0 \dots 2.0)$ balról zárt intervallumba normált szám együtttható része
- ezt az alakot a valós szám un. normalizált valós alakjának nevezzük
- az elkülönülő előjel miatt a mantissza mindig pozitív
- a mantissza egész része az intervallumból adódóan mindig 1, így ezt a szabvány előírásának megfelelően nem is ábrázoljuk (*rejtett 1-esnek hívjuk*)
- fraction mezőben ábrázolt mantissza (f db bit) tehát valójában a tényleges mantissza törtrésze ($m = 1.\text{törtrész}$)

A lebegőpontos számábrázolás

$$x = \pm m \cdot 2^k,$$

Az exponens (k):

- a "valódi" kitevő (karakterisztika) 0 értékéhez a mező méretéből adódó (érték-1) tartozik (pl. 8 bit esetében 127)
- ezt a nullponteltolást offsetnek nevezzük
- e db bit
- az exponens ábrázolásából adódó minimális (0) és maximális értékeket speciális célokra tartották fent

A lebegőpontos számábrázolás - példa #1

$$x = \pm w \cdot 2^k$$

$$k = 2$$

$$w = 5/4 = 1.25$$

Legyen $x = 5$, amit ha az $[1, \dots, 2)$ intervallumba normálunk, akkor $x = +1.25 \cdot 2^2$

- a keletkező szám előjele (s) : $+(=0)$ ✓
- az exponens értéke $2(+127 \stackrel{2^3-1}{=} 129)$
- a mantissza binárisan 1.01_2 , amelyből a rejtett 1-es levágása után $.01_2$ marad

$$1.25 \rightarrow 1.01$$

Példa az $x = 5$ egyszeres pontosságú ábrázolásra:

s	karakterisztika								mantissza																		
0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1. bájt									2. bájt				3. bájt				4. bájt										

Innen már hexadecimálisan is könnyen felírható: 40A00000₁₆.

A lebegőpontos számábrázolás - feladat, példa #2

$$x = \pm m \cdot 2^k$$

Legyen $x = -0.75$, amit ha az $[1, \dots, 2)$ intervallumba normálunk, akkor $x = -1,5 \cdot 2^{-1}$

- a keletkező szám előjele (s) : $- : 1$
- az exponens értéke $-1 + 127 = 126$
- a mantissza binárisan: $1,5_{10} \rightarrow 1,1_2$
- amelyből a rejtett 1-es levágása után marad : $,1$

Példa az $x = -0.75$ egyszeres pontosságú ábrázolásra:

s	karakterisztika								mantissza															
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
B F									4								3. bájt				4. bájt			

Innen már hexadecimálisan is könnyen felírható: $BF40...0_{16}$

A lebegőpontos számábrázolás - feladat, példa #3

$$x = \pm m \cdot 2^k$$

Legyen $x = \frac{1}{3}$, amit ha az $[1, \dots, 2)$ intervallumba normálunk, akkor $x = \frac{1}{3} \cdot 2^{-2}$

- a keletkező szám előjele (s) : $+$ = 0
- az exponens értéke: $-2 + 127 = 125$
- a mantissza binárisan: $1, 010101 \dots$
amelyből a rejtett 1-es levágása után marad: $, 010101 \dots$

Példa az $x = \frac{1}{3}$ egyszeres pontosságú ábrázolásra:

s	karakterisztika								mantissza																
0	0	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0
3			1. bájt		E		2. bájt				3. bájt				4. bájt										

Innen már hexadecimálisan is könnyen felírható: $3EA A \dots A$

